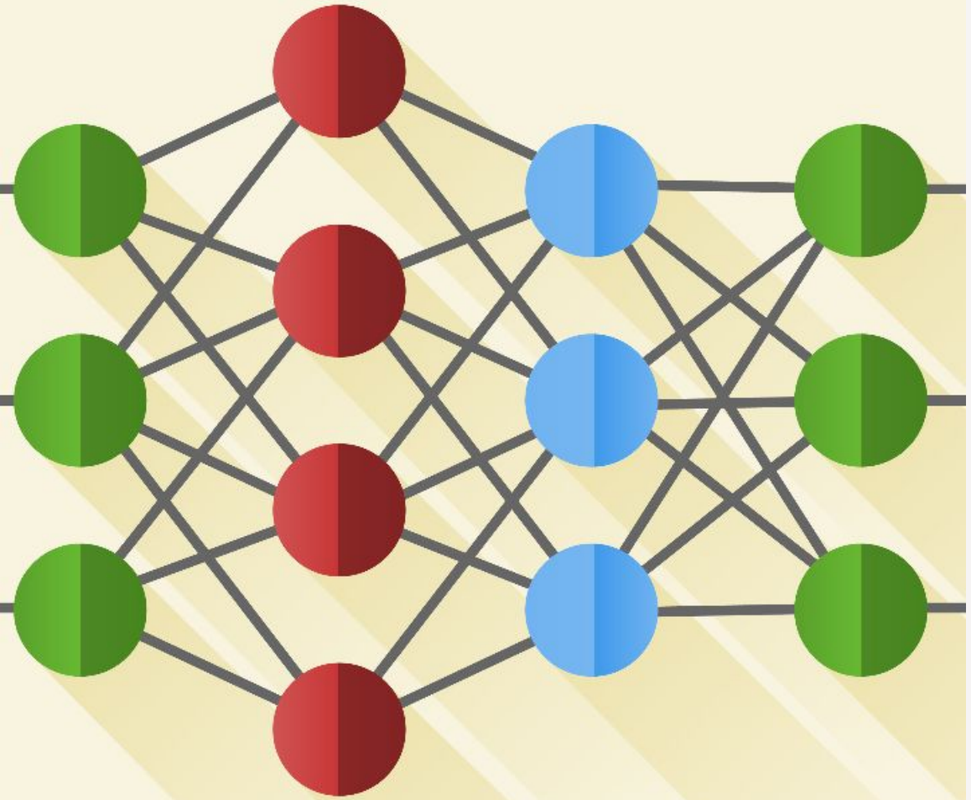


Convolutional Neural Networks (CNN) — Architecture

.

C

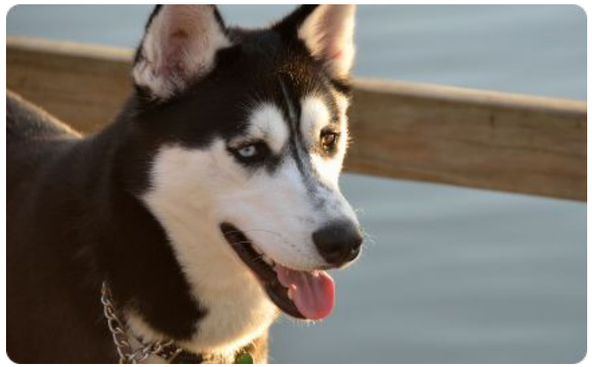
by CHINCHU

M

MS

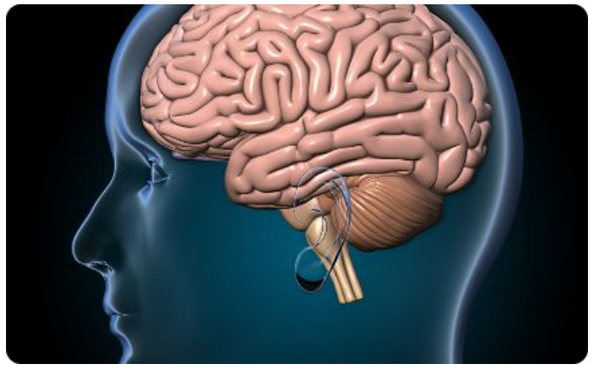
- A convolutional neural network (CNN), is a network architecture for deep learning which learns directly from data.
- CNNs are particularly useful for finding patterns in images to recognize objects.
- They can also be quite effective for classifying non-image data such as audio, time series, and signal data.
- A Convolutional Neural Network (CNN) is a specialized type of deep learning model that is primarily used for image recognition and analysis.
- It consists of multiple interconnected layers that perform convolutions, pooling, and feature extraction.
- CNNs are highly effective in pattern recognition tasks due to their ability to automatically learn and extract relevant features from images.

CNN Basics



Introduction

CNNs are a type of AI model commonly used for image processing and recognition.



Neural Networks

CNNs are inspired by the structure of the brain and its ability to process visual information.

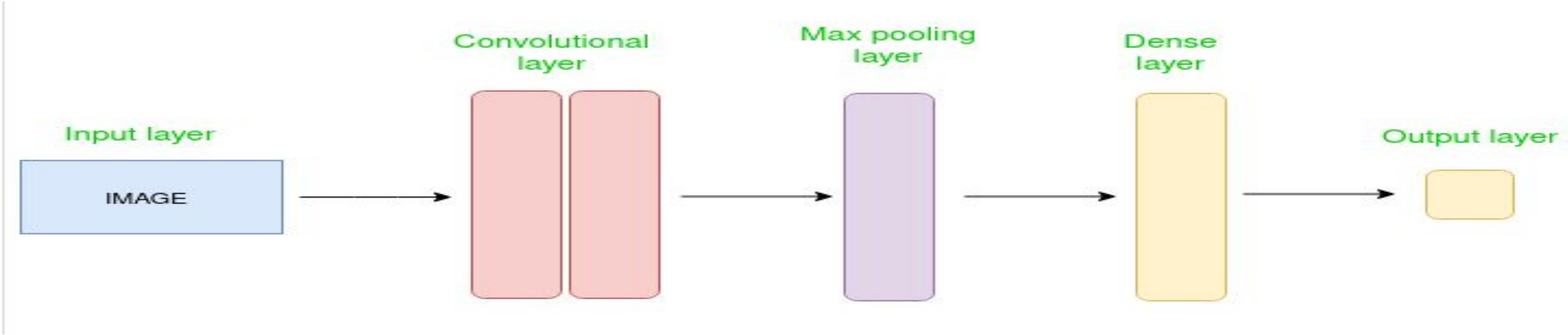


Mathematics

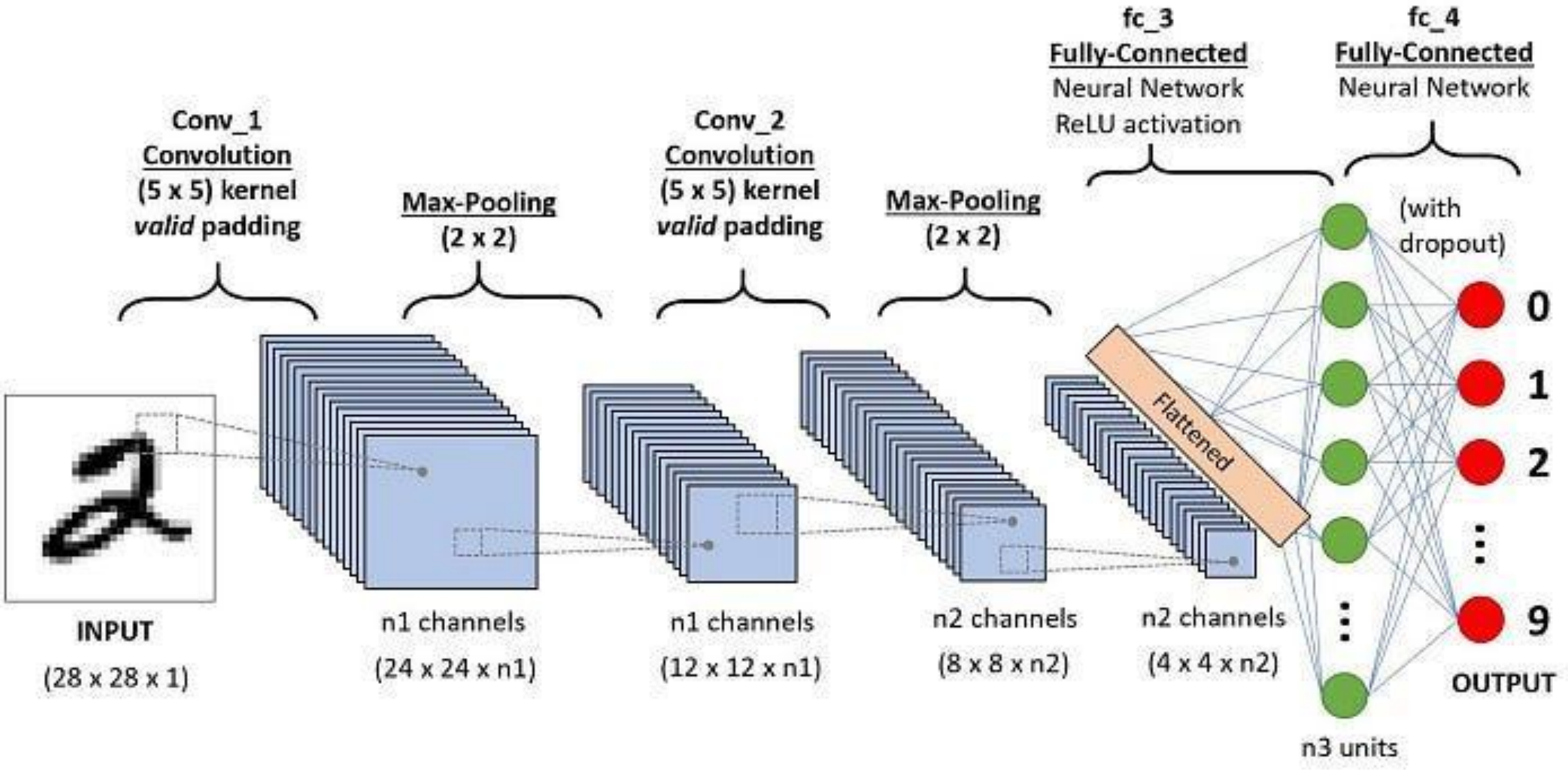
CNNs use mathematical operations such as convolution and pooling to extract features from images.

CNN architecture

- Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers



- The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction.
- The network learns the optimal filters through backpropagation and gradient descent.

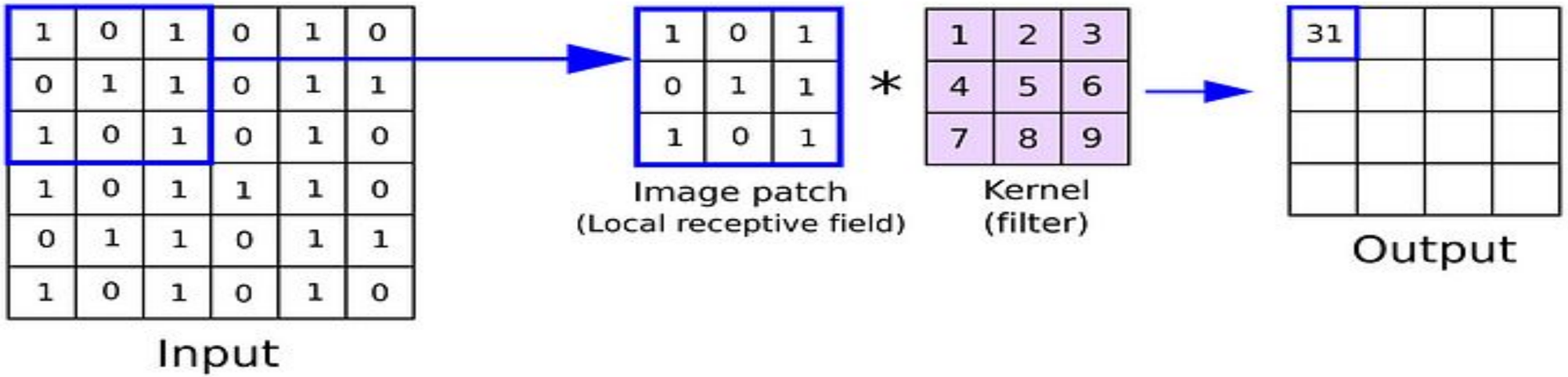

Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with dropout)
Flattened
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
n3 units
0
1
2
9
OUTPUT

Kernel or Filter or Feature Detectors

In a convolutional neural network, the kernel is nothing but a filter that is used to extract the features from the images.

Formula = [i-k]+1

i -> Size of input , K-> Size of kernel

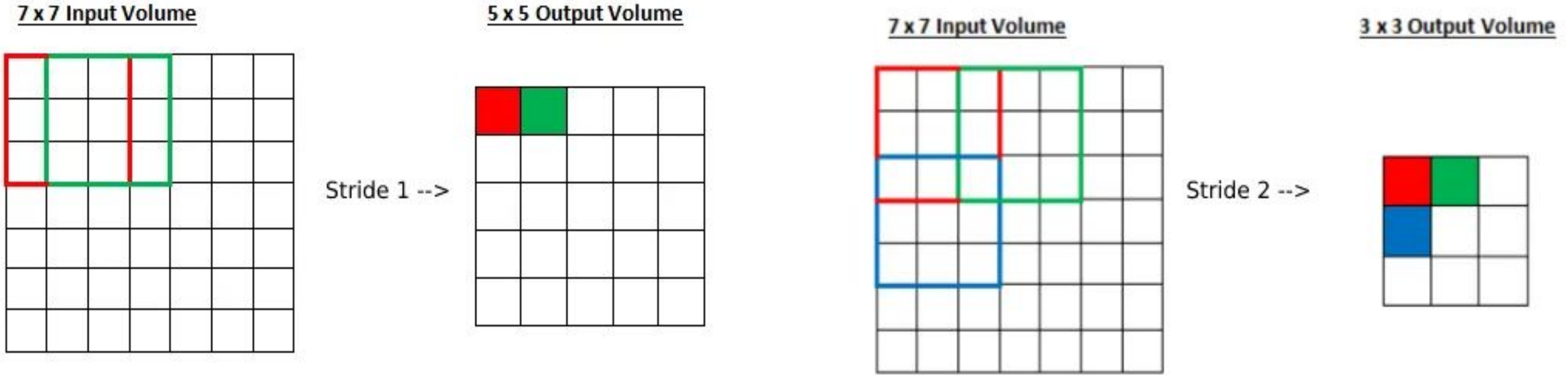


Stride

Stride is a parameter of the neural network's filter that modifies the amount of movement over the image or video. we had stride 1 so it will take one by one. If we give stride 2 then it will take value by skipping the next 2 pixels.

Formula =[i-k/s]+1

i -> Size of input , K-> Size of kernel, S-> Stride

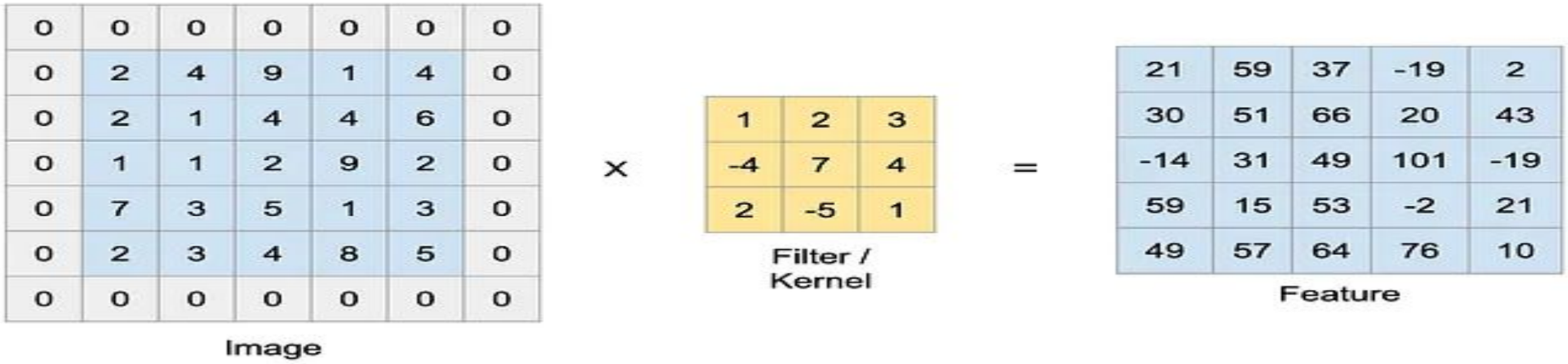


Padding

Padding is a term relevant to convolutional neural networks as it refers to the number of pixels added to an image when it is being processed by the kernel of a CNN. For example, if the padding in a CNN is set to zero, then every pixel value that is added will be of value zero. When we use the filter or Kernel to scan the image, the size of the image will go smaller. We have to avoid that because we want preserve the original size of the image to extract some low-level features. Therefore, we will add some extra pixels outside the image. Kindly use this link to learn more about padding.

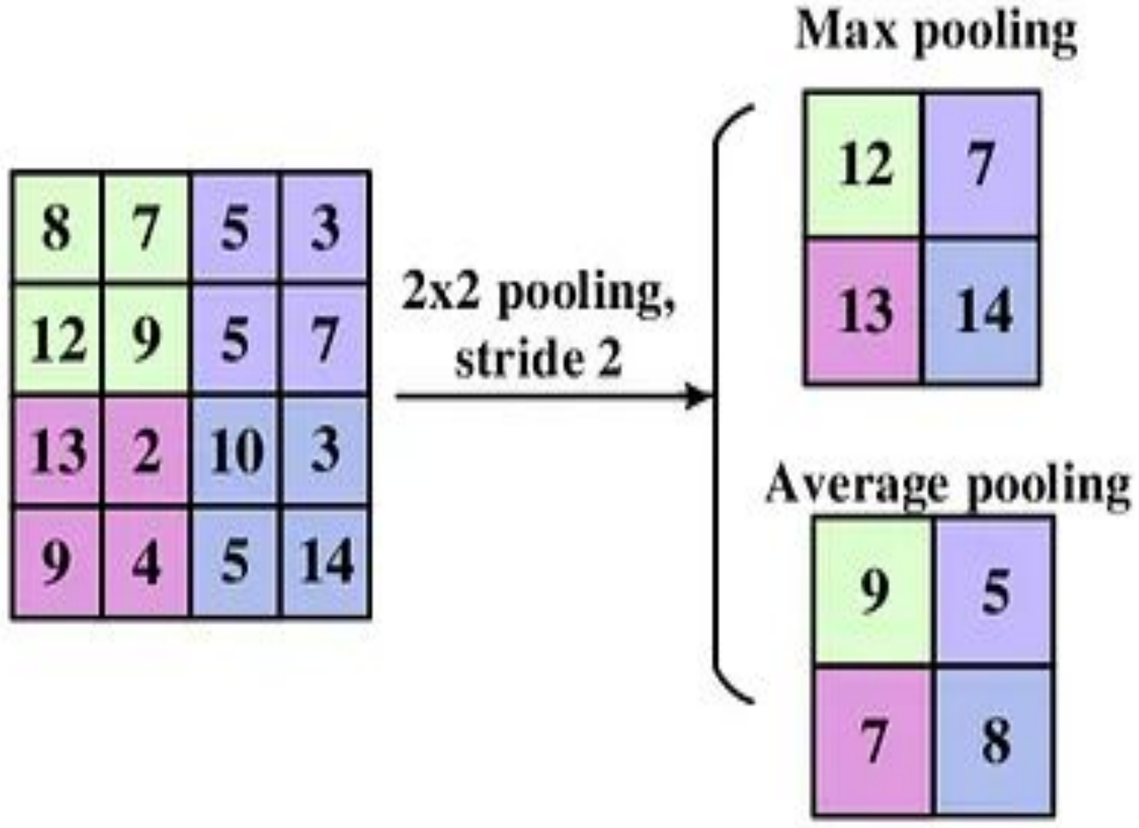
Formula =[i-k+2p/s]+1

i -> Size of input , K-> Size of kernel, S-> Stride, p->Padding



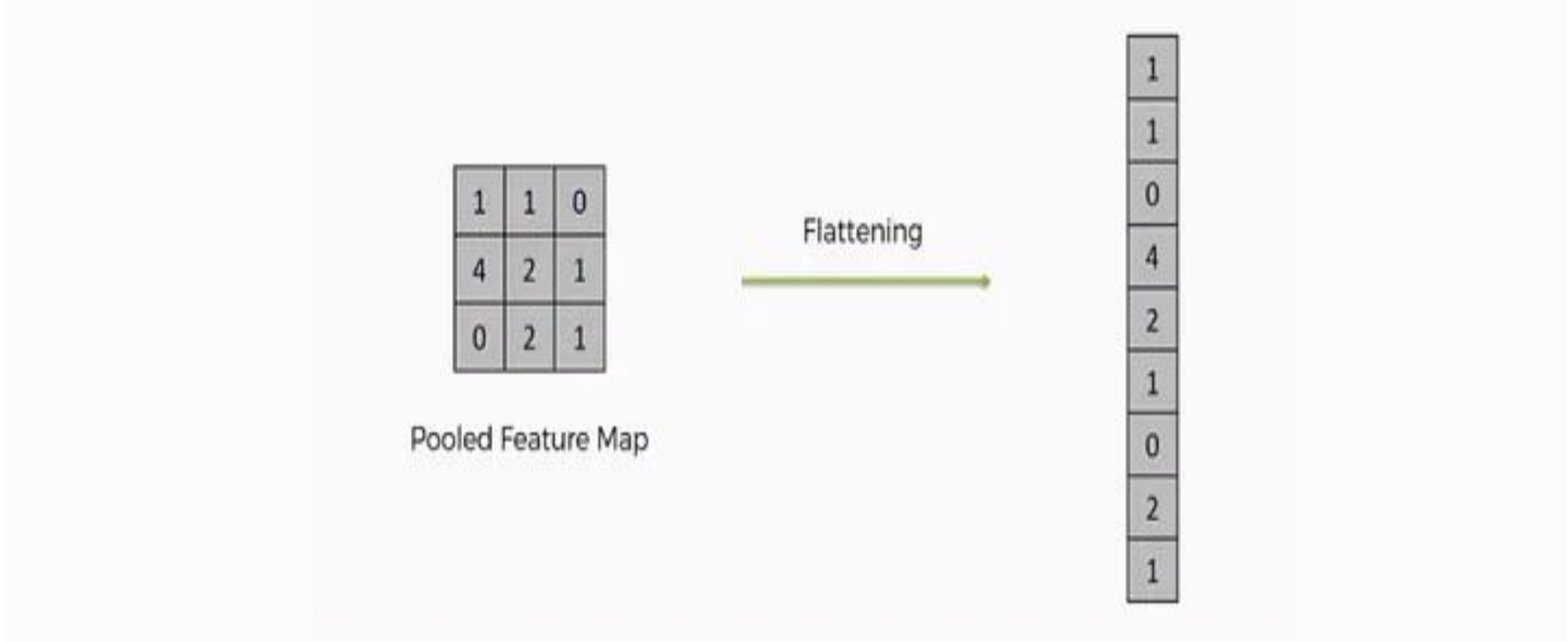
Pooling

Pooling in convolutional neural networks is a technique for generalizing features extracted by convolutional filters and helping the network recognize features independent of their location in the image.



Flatten

Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.



Layers used to build CNN

Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:

Convolutional layer
Pooling layer
Fully-connected (FC) layer

Convolutional layer

This layer is the first layer that is used to extract the various features from the input images. In this layer, We use a filter or Kernel method to extract features from the input image.

Pooling layer

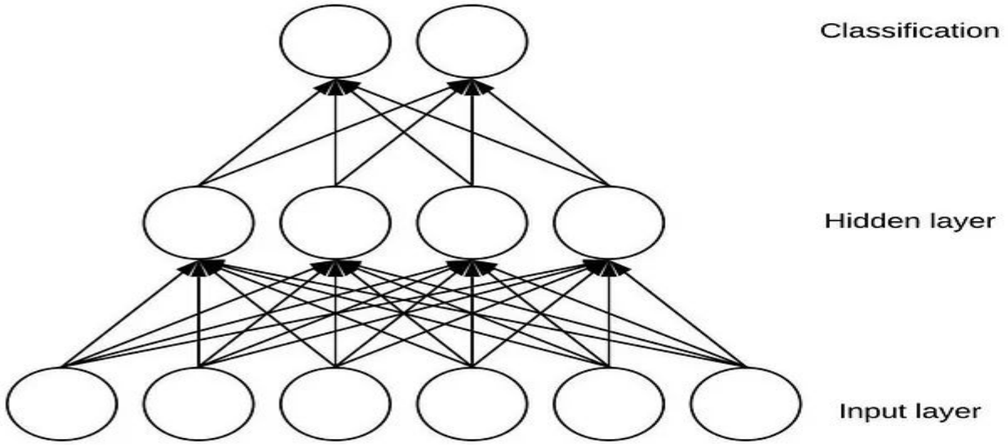
The primary aim of this layer is to decrease the size of the convolved feature map to reduce computational costs. This is performed by decreasing the connections between layers and independently operating on each feature map. Depending upon the method used, there are several types of Pooling operations. We have Max pooling and average pooling.

Fully-connected layer

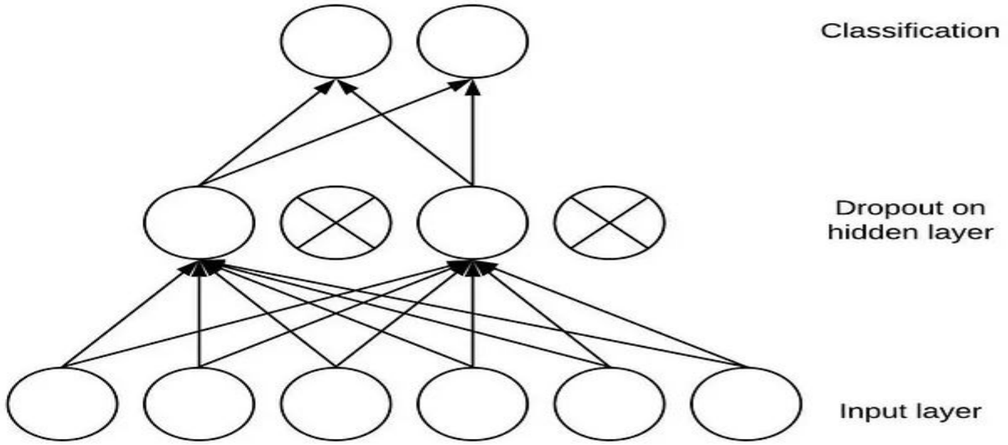
The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. These layers are usually placed before the output layer and form the last few layers of a CNN Architecture.

Dropout

Another typical characteristic of CNNs is a Dropout layer. The Dropout layer is a mask that nullifies the contribution of some neurons towards the next layer and leaves unmodified all others.



Without Dropout



With Dropout

Activation Function

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction. There are several commonly used activation functions such as the ReLU, Softmax, tanH, and the Sigmoid functions. Each of these functions has a specific usage.

Sigmoid — For a binary classification in the CNN model

tanH - The tanh function is very similar to the sigmoid function. The only difference is that it is symmetric around the origin. The range of values, in this case, is from -1 to 1.

Softmax- It is used in multinomial logistic regression and is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.

RelU- the main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

Convolutional Layers

Purpose

Convolutional layers apply filters to an image and create feature maps that reduce the dimensionality of the image.

Operations

Convolutions involve multiplying a small matrix (the filter) with each pixel of the image and summing the results.

Filters and Feature Maps

Filters are small feature detectors that are applied to the image. Feature maps show the locations of the detected features.

Examples

Applications of convolutional layers include image classification, object detection, and image segmentation.

Pooling Layers

1

Importance

Pooling layers reduce the size of the feature maps and make them more robust to small variations in the image.

2

Techniques

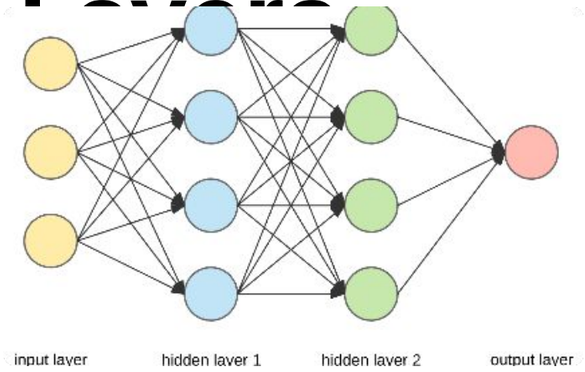
Common pooling techniques include max pooling (selecting the maximum value in each region), average pooling (averaging the values in each region), and L2 pooling (taking the square root of the sum of squares of the values in each region).

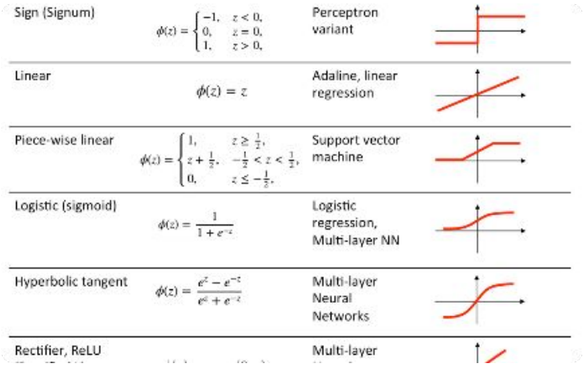
3

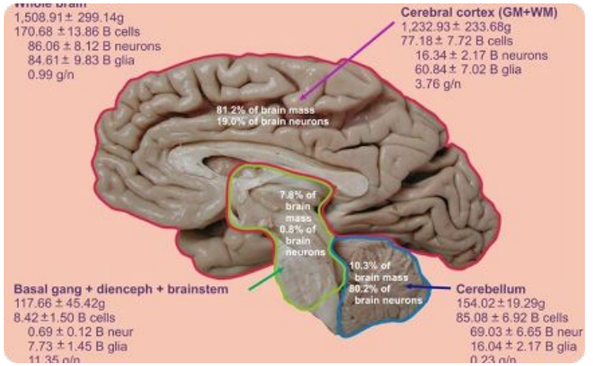
Downsampling

Pooling layers perform downsampling, which reduces the dimensionality of the network and helps prevent overfitting.

Fully Connected Layers







Role

Fully connected layers connect all the neurons in one layer to all the neurons in the next layer. They perform the final classification or regression of the input data.

Activation Functions

Activation functions introduce non-linearity into the model and allow it to learn complex representations of the input data.

Number of Neurons

The number of neurons in each layer affects the complexity and capacity of the model. Determining the optimal number of neurons requires experimenting and testing.

Advantages of Convolutional Neural Networks (CNNs):

1. Good at detecting patterns and features in images, videos, and audio signals.
2. Robust to translation, rotation, and scaling invariance.
3. End-to-end training, no need for manual feature extraction.
4. Can handle large amounts of data and achieve high accuracy.

Disadvantages of Convolutional Neural Networks (CNNs):

1. Computationally expensive to train and require a lot of memory.
2. Can be prone to overfitting if not enough data or proper regularization is used.
3. Requires large amounts of labeled data.
4. Interpretability is limited, it’s hard to understand what the network has learned.

Efficient Convolution Algorithm

1D Discrete Convolution:

Step 1: Understanding the Problem

In 1D discrete convolution, you have two sequences: f (the input signal) of length N, and g (the filter/kernel) of length M. The convolution of f and g, denoted as $f * g$, results in another sequence h of length $N + M - 1$. The ith element of h is calculated by summing the element-wise products of f and a reversed and shifted version of g. Mathematically, it can be represented as:

$$h_i = \sum_{j=0}^{M-1} f_{i-j} \times g_j$$

Step 2: Padding (Optional but often used for boundary handling)

Padding involves adding zeros to the input sequence f to handle convolution near the boundaries. This step ensures that the output sequence h has the same length as f. There are different padding strategies (zero-padding, reflection-padding, etc.) based on the specific problem requirements.

Step 3: Flip the Kernel

Reverse the filter/kernel g to get g'. For example, if $g = [a, b, c]$, then $g' = [c, b, a]$.

Step 4: Convolution Operation

Slide the flipped kernel g' over the input sequence f, multiplying and summing the overlapping elements to obtain the elements of the output sequence h. This process is often referred to as the "sliding window" operation.

- For $i = 0$ to $N + M - 2$:
  - $h_i = 0$
  - For $j = \max(0, i - N + 1)$ to $\min(i, M - 1)$:
    - $h_i += f_{i-j} \times g'_j$

Repeat this process until you have calculated all elements of h.

Step 5: Output

The resulting sequence h is the convolution of f and g.

This basic algorithm has a time complexity of $O(N \times M)$, where N is the length of the input sequence and M is the length of the kernel. There are more advanced algorithms and techniques, such as fast Fourier transform (FFT) based convolution, which can significantly reduce the computational complexity for large sequences. These methods are more complex but are essential for real-time applications or when dealing with large datasets.

2D convolution is a fundamental operation in image processing and computer vision. It involves convolving a 2D input image matrix with a 2D kernel (also known as a filter or mask) to produce a 2D output image. The convolution operation is used for various tasks such as edge detection, blurring, sharpening, and feature extraction in images.

Step 1: Understanding the Problem

In 2D convolution, you have a 2D input matrix I of size $N \times M$ representing the input image, and a 2D kernel matrix K of size $P \times Q$. The convolution of I and K, denoted as $I * K$, results in another 2D matrix O of size $(N - P + 1) \times (M - Q + 1)$.

Step 2: Padding (Optional but often used for boundary handling)

Similar to 1D convolution, you can optionally pad the input matrix I with zeros to handle convolution near the boundaries. Padding ensures that the output matrix O has the desired dimensions.

Re

Step 3: Flip the Kernel

Reverse the 2D kernel matrix K both horizontally and vertically to get K'.

Step 4: Convolution Operation

Slide the flipped kernel K' over the input matrix I, element-wise multiplying and summ the overlapping elements to obtain the elements of the output matrix O. This process i similar to the 1D sliding window operation but is extended to 2D.

- For $i = 0$ to $N - P$:
  - For $j = 0$ to $M - Q$:
    - $O[i][j] = \sum_{a=0}^{P-1} \sum_{b=0}^{Q-1} I[i + a][j + b] \times K'[a][b]$

Repeat this process until you have calculated all elements of O.

Step 5: Output

The resulting matrix O is the 2D convolution of the input image I and the kernel K.

It's important to note that different applications might require different types of kernels. For example, a kernel for edge detection might look for changes in intensity, while a kernel for blurring might average the pixel values in the neighborhood.

Efficient algorithms for 2D convolution often use techniques like **separable convolution** (where a 2D convolution is split into two 1D convolutions) and **fast Fourier transform (FFT)** based methods, especially for large images and kernels. These techniques help reduce the computational complexity and speed up the convolution process.